Clase 36 · Boosting

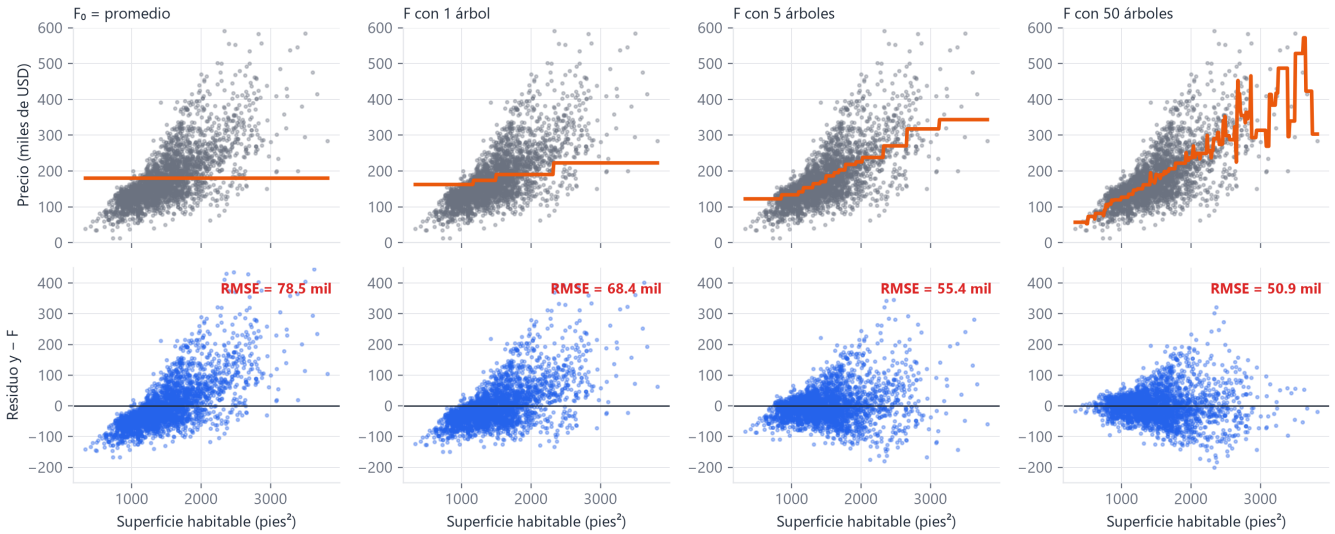
Profesor Bastian Galasso · Etapa 7 del roadmap · continúa Árboles (C34) y Random Forest (C35)

La gran idea de la clase

Un bosque entrena muchos árboles **independientes** y los promedia. Boosting hace lo contrario: entrena los árboles **uno tras otro** y cada uno se concentra en lo que los anteriores no lograron explicar. «Si un modelo se equivocó en una parte de los datos, queremos que el siguiente aprenda de ese error y no tropiece dos veces con la misma piedra.» El resultado es un modelo **aditivo**: la suma ponderada de todos los árboles.

Boosting en una imagen: ajustar residuos

Cada árbol se ajusta a lo que el modelo anterior no explicó: los residuos se achican (η = 0.3, profundidad 2)



Lo que necesitas ANTES

- Árboles de decisión: profundidad, hojas, sobreajuste (Clase 34)
- Bagging y Random Forest: por qué promediar reduce la varianza (Clase 35)
- Validación cruzada, AUC y RMSE (Clases 30–33)
- Pipeline con imputación y one-hot (preprocesamiento dentro del pipeline)

Ruta de la clase y datos

- **Intuición** → **AdaBoost** (pesos) → **Gradient Boosting** (gradientes) → **XGBoost y LightGBM** → comparar y elegir (CatBoost solo se menciona)
- **Heart** (303 pacientes, clasificación AHD) y **Ames Housing** (2.930 casas, regresión del precio): los mismos datos de la Clase 35
- Pregunta guía: ¿cómo equilibramos capacidad predictiva, velocidad, categóricas e interpretabilidad?

Clase 36 · Bagging vs boosting y el ciclo general

Fuente: transcripción de la clase + diapositivas + notebook Python del profesor

Los dos son ensambles de árboles, pero resuelven problemas distintos. Bagging promedia modelos independientes para bajar la varianza; boosting los encadena para bajar el sesgo y el error.

Conceptos clave

1. **Paralelo vs secuencial.** En el bosque cada árbol se entrena solo, con su muestra, «sin importarle el de al lado». En boosting el árbol 2 necesita el feedback del árbol 1: no se puede paralelizar entre árboles.
2. **Mismo dataset completo.** Cada etapa ve todos los datos de entrenamiento. El error del árbol anterior viaja como pesos (AdaBoost) o como residuos (Gradient Boosting); remuestrear lo destruiría.
3. **Suma, no promedio.** El modelo final es aditivo: $F(x) = F_0 + \eta \cdot h_1(x) + \eta \cdot h_2(x) + \dots$ La contribución de cada árbol se pondera; en AdaBoost, según su calidad.
4. **Ciclo de 5 pasos.** 1) Inicializar con una predicción base · 2) medir el error (pérdida o casos difíciles) · 3) ajustar un nuevo aprendiz · 4) sumar su contribución · 5) detener tras M etapas o cuando la validación deja de mejorar.
5. **Lo que se espera del árbol 2.** No que cometa menos errores en todo, sino que **no cometa los mismos**: se concentra en lo difícil y descuida un poco lo que ya aprendió bien.



Izquierda: el esquema de cada familia. Derecha: error en validación de Ames al agregar árboles. El bosque se estanca; boosting sigue mejorando.

Fórmulas y reglas

$F_M(x) = F_0(x) + \eta \cdot \Sigma\, h_m(x)$, m = 1…M

bagging → paralelo, promedio o voto, ↓ varianza

boosting → secuencia, suma ponderada, ↓ sesgo y error

tiempo con 1.000 árboles y 10 núcleos: bagging ≈ 100 · boosting ≈ 1.000

En Python

```
RandomForestRegressor(n_estimators=300,
    n_jobs=-1) # paraleliza el bosque
GradientBoostingRegressor(n_estimators=300,
    learning_rate=0.1, max_depth=3)
list(gb.staged_predict(X_val)) # F₁, F₂, …
```

Con los datos de la clase

- Árbol solo (profundidad 3): RMSE de validación 41.2k USD
- Bosque de 300 árboles: 23.1k, ya estancado hacia los 100 árboles
- Gradient Boosting de 300 árboles (η = 0.1): 21.8k, mejor ronda ≈ 250
- Con un solo árbol el boosting va peor (69.0k) a propósito: da pasos chicos

Cuidado
Por ser secuencial, boosting es más sensible a **ruido**, hiperparámetros y **filtración** de información: una etiqueta mal puesta se convierte en el caso «más difícil» y el modelo se obsesiona con ella.

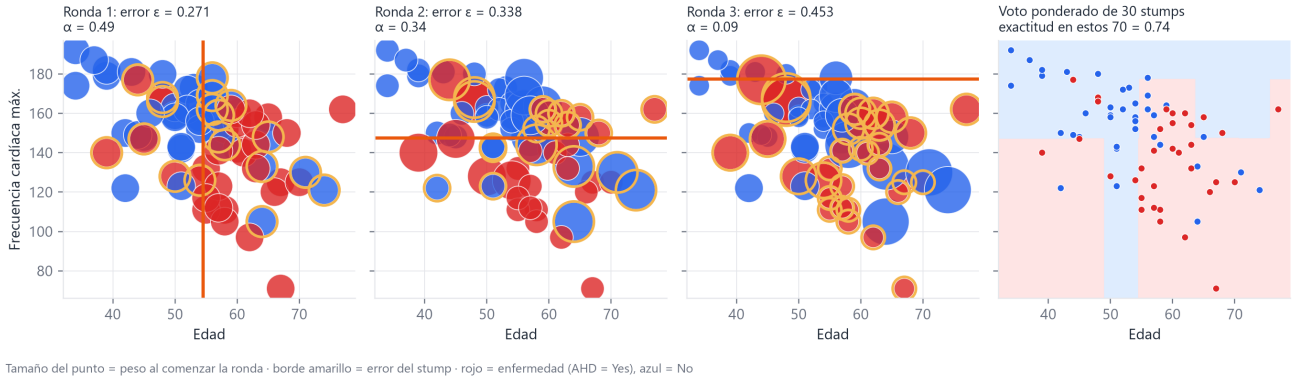
Clase 36 · AdaBoost: enfocar los casos difíciles

Fuente: transcripción de la clase + diapositivas + notebook Python del profesor

AdaBoost sube el peso de las observaciones mal clasificadas, de modo que el siguiente árbol (normalmente un stump de un solo corte) las tome más en serio, y combina los árboles con un voto ponderado por su calidad.

Conceptos clave

1. **Pesos iguales al inicio.** $w_i = 1/n$. El árbol 1 se entrena con todos los pesos iguales.
2. **Error del modelo ε.** Suma de los pesos de los casos mal clasificados, entre la suma de todos los pesos: es un número entre 0 y 1. Perfecto: 0; errar todo: 1.
3. **Peso del modelo α.** $\alpha = \frac{1}{2} \ln((1 - \varepsilon)/\varepsilon)$. Con $\varepsilon = 0.5$ (azar) $\alpha = 0$: no aporta. Con $\varepsilon > 0.5$, $\alpha < 0$. «Mayor error del árbol, menor influencia.»
4. **Reponderar.** Los casos mal clasificados multiplican su peso por e^{α}; los bien clasificados, por $e^{-\alpha}$. Luego se normaliza para que sumen 1.
5. **Voto final.** $H(x) = \text{signo de } \Sigma\, \alpha_m \cdot h_m(x)$, con clases −1/+1. Cada stump corrige una región; el voto produce una frontera «escalonada».



Con 70 pacientes de Heart y dos variables (edad y frecuencia cardíaca máxima). Cada ronda agranda los puntos mal clasificados.

Fórmulas y reglas

$$\varepsilon_m = \Sigma\, w_i \cdot 1[h_m(x_i) \neq y_i] \,/\, \Sigma\, w_i$$

$$\alpha_m = \tfrac{1}{2} \cdot \ln((1 - \varepsilon_m) / \varepsilon_m)$$

$$w_i \leftarrow w_i \cdot \exp(-\alpha_m \cdot y_i \cdot h_m(x_i)), \text{ luego normalizar}$$

$$H(x) = \text{sign}(\, \Sigma\, \alpha_m \cdot h_m(x)\,)$$

En Python

```
AdaBoostClassifier(n_estimators=250,
    learning_rate=0.05, random_state=42)
# estimador base por defecto: stump (prof. 1)
AdaBoostClassifier(estimator=
    DecisionTreeClassifier(max_depth=2))
```

Con los datos de la clase

- Rondas 1 a 3: ε = 0.271 → 0.338 → 0.453; α = 0.49 → 0.34 → 0.09
- A medida que quedan solo casos difíciles, ε se acerca a 0.5 y α a 0
- Heart (CV 5×3): AdaBoost 0.912 de AUC, el mejor de los cinco, pero a menos de un desvío (≈ 0.04) del último
- Ames: AdaBoost es el peor (RMSE 36.2k vs 24.4k de XGBoost)

Cuidado
AdaBoost es «Adaptive Boosting»; en clase se dijo «additive». Es sensible a ruido y etiquetas incorrectas: úsalo como línea base fuerte con datos limpios.

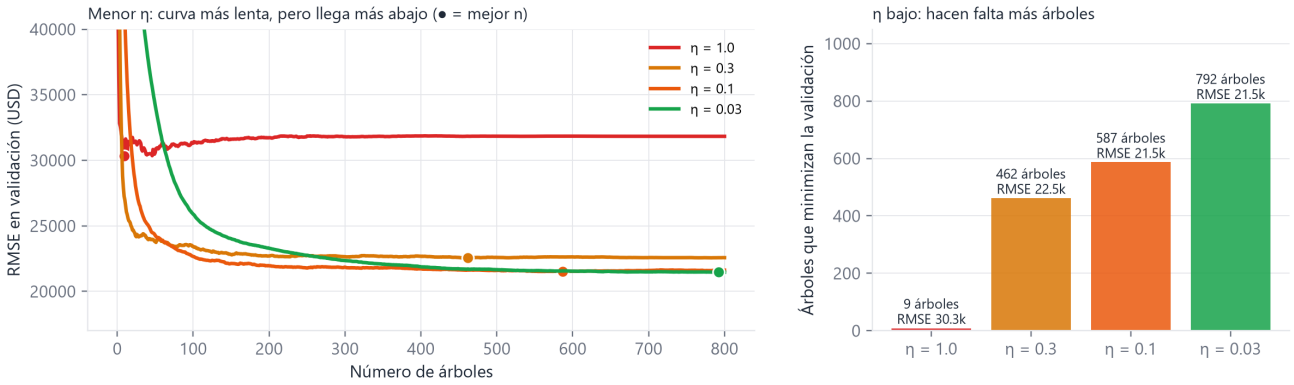
Clase 36 · Gradient Boosting y las tres perillas

Fuente: transcripción de la clase + diapositivas + notebook Python del profesor

En vez de pesar casos, Gradient Boosting ajusta cada árbol a la dirección que más reduce la pérdida. Con error cuadrático, esa dirección son exactamente los residuos y − F. Tres perillas controlan todo: pérdida, tasa η y complejidad del árbol.

Conceptos clave

1. **La pérdida define el error.** Cuadrática, absoluta, Huber o logística: es la forma en que un árbol le dice al siguiente qué corregir. Otra pérdida, otro resultado.
2. **Gradiente negativo = residuo.** El nuevo árbol h_m predice $-\partial L/\partial F$. Para $L = \frac{1}{2}(y - F)^2$ eso es $y - F$; para otras pérdidas ya no es el residuo a secas.
3. **Tasa de aprendizaje η.** Cuánto de cada árbol se suma. Menor η suele requerir más árboles: «estudiar un poco cada día» rinde más que de golpe la noche anterior.
4. **η y n_estimators van juntos.** Más árboles con η bajo da modelos más estables, pero más caros. Se afinan en pareja, nunca solos.
5. **Complejidad del árbol.** max_depth, número de hojas, muestras mínimas por hoja: limitan lo que aprende cada árbol (los árboles de boosting suelen ser chicos, de profundidad 2 a 8).



Gradient Boosting con profundidad 3 sobre Ames, validación separada del test. Los puntos marcan el número de árboles óptimo para cada η.

Fórmulas y reglas

$F_m = F_{m-1} + \eta \cdot h_m$

$r_{im} = -\partial L/\partial F$ evaluado en F_{m-1} (cuadrática: $y - F$)

caso ideal: residuo tras m pasos = $r_0 \cdot (1 - \eta)^m$

con η = 1 cada árbol corrige todo su residuo (¡y su ruido!)

En Python

```
GradientBoostingRegressor(
    n_estimators=500, learning_rate=0.05,
    max_depth=2, loss='squared_error')
# loss: 'huber', 'absolute_error' (regresión)
# clasificación: log_loss
```

Con los datos de la clase

- η = 1.0: mejor a los 9 árboles, RMSE 30.3k; luego empeora
- η = 0.3: 462 árboles, 22.5k
- η = 0.1: 587 árboles, 21.5k
- η = 0.03: ≈ 790 árboles (casi el tope de 800), 21.5k: igual de bueno, mucho más lento

Cuidado
Con η = 0.03 y pocos árboles el modelo queda **subajustado**. «A menor η, mejor» solo vale si das árboles suficientes. Y los residuos son el gradiente solo con pérdida cuadrática.

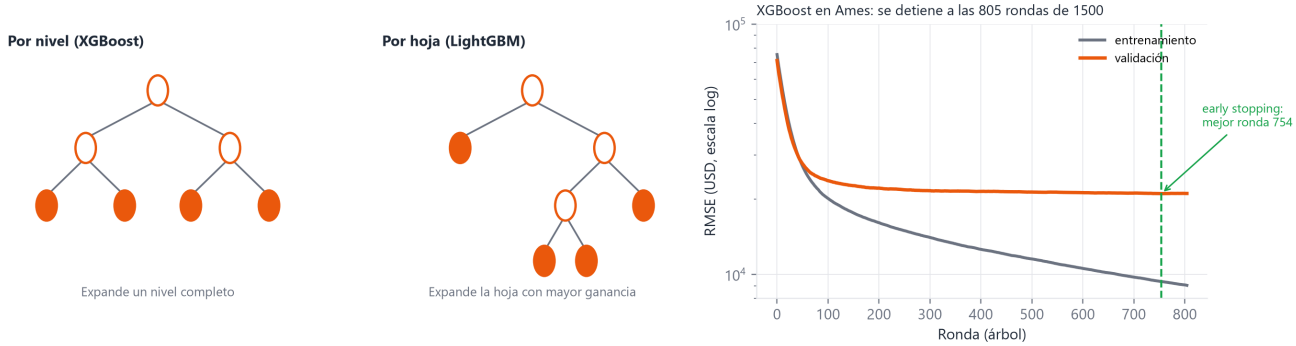
Clase 36 · XGBoost y LightGBM

Fuente: transcripción de la clase + diapositivas + notebook Python del profesor

Las implementaciones modernas son Gradient Boosting con ingeniería: XGBoost agrega regularización y velocidad; LightGBM acelera la búsqueda de cortes con histogramas y crece el árbol hoja por hoja.

Conceptos clave

1. **XGBoost: objetivo regularizado.** Pérdida de entrenamiento + penalización por complejidad (premia la simplicidad). Usa gradientes de 1.er y 2.º orden, submuestreo de filas y columnas, manejo eficiente de dispersión y de nulos. Está escrito en C++ y se llama desde Python.
2. **LightGBM: histogramas.** Agrupa los valores continuos en bins y busca los cortes ahí: menos memoria y tiempo. GOSS conserva gradientes grandes y muestrea los pequeños; EFB agrupa variables dispersas casi excluyentes.
3. **Por nivel vs por hoja.** XGBoost suele expandir un nivel completo. LightGBM expande solo la hoja con **mayor ganancia**: árbol asimétrico, mucho más eficiente con muchos datos.
4. **Riesgo del leaf-wise.** Puede sobreajustar datos pequeños si no se limita (num_leaves, min_data_in_leaf, max_depth). LightGBM está pensado para muchos datos.
5. **Early stopping.** Se monitorea el error en un conjunto de validación y se detiene cuando deja de mejorar durante k rondas: elige n_estimators automáticamente.



Izquierda: dos formas de hacer crecer un árbol. Derecha: XGBoost con early stopping en Ames; el entrenamiento sigue bajando, la validación se aplana.

Fórmulas y reglas

$\text{Obj} = \Sigma\, L(y_i, \hat{y}_i) + \Sigma\, \Omega(f_k)$

$\Omega(f) = \gamma \cdot T + \tfrac{1}{2} \cdot \lambda \cdot \Sigma\, w_j^2$ (T hojas, λ = L2)

hojas máximas con profundidad d: 2^d

LightGBM: num_leaves < $2^{\text{max_depth}}$

En Python

```
XGBRegressor(n_estimators=1500,
    learning_rate=0.05, max_depth=3,
    subsample=0.8, colsample_bytree=0.8,
    early_stopping_rounds=50)
LGBMClassifier(num_leaves=15,
    subsample=0.8, subsample_freq=1)
```

Con los datos de la clase

- XGBoost con early stopping en Ames: mejor ronda 754, se detiene en la 805 de 1.500 posibles
- En esa ronda: RMSE de validación 21.0k y de entrenamiento 9.4k
- Tiempos con 350 árboles en Ames: XGBoost 0.5 s, LightGBM 0.6 s, Gradient Boosting de sklearn 3.2 s, AdaBoost 6.3 s
- LightGBM (31 hojas) en Ames: RMSE de entrenamiento 8.0k vs test 27.0k: sobreajuste

Cuidado
El early stopping necesita su propio conjunto de validación, **nunca el test**. En LightGBM, `subsample` no hace nada mientras `subsample_freq` sea 0 (el valor por defecto).

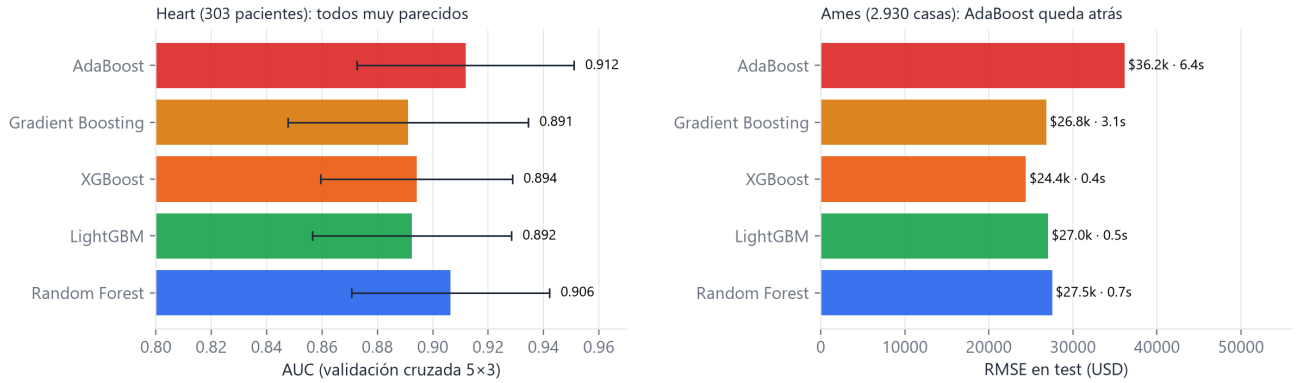
Clase 36 · Comparar y elegir

Fuente: transcripción de la clase + diapositivas + notebook Python del profesor

No hay un ganador universal. La mejor opción depende del tamaño de los datos, las categóricas, el presupuesto de cómputo y, sobre todo, de la validación: mismo pipeline, misma partición, métrica adecuada.

Conceptos clave

1. **Qué domina tu problema.** Línea base simple: AdaBoost · control didáctico: Gradient Boosting · rendimiento general: XGBoost · gran escala: LightGBM · muchísimas categóricas: CatBoost.
2. **Categóricas.** AdaBoost, Gradient Boosting y (normalmente) XGBoost piden codificarlas; LightGBM las admite de forma nativa; CatBoost es el especialista.
3. **«A problema simple, modelo simple».** Con pocos datos (Heart, 303 filas) los modelos simples compiten con los avanzados. Con más datos (Ames, 2.930) XGBoost y los de gradiente despegan.
4. **Mide bien.** Clasificación: AUC, F1, recall (¿cuánto cuesta un falso negativo?). Regresión: RMSE, MAE y R^2. Compara train y test para ver el sobreajuste, y reserva el test para el final.
5. **Dónde se usan.** Riesgo (impago, fraude, churn), demanda, salud, marketing, operaciones, precios: datos tabulares con no linealidades e interacciones.



Los cinco modelos con la configuración del notebook del profesor (más Random Forest como referencia). Heart: validación cruzada 5×3. Ames: partición de test.

Fórmulas y reglas

AUC con CV: reportar media ± desvío

RMSE = √(promedio de $(\hat{y} - y)^2$) · en la misma unidad que y

1 paciente de 76 en test = 1.3 puntos de exactitud

ganador = diferencia mayor que el desvío entre rondas

En Python

```
Pipeline([('prep', ColumnTransformer(...)),
          ('modelo', XGBClassifier(...))])
cross_val_score(pipe, X, y, cv=cv,
    scoring='roc_auc')
scoring='neg_root_mean_squared_error'
```

Con los datos de la clase

- Heart, AUC (CV 5×3): AdaBoost 0.912 · RF 0.906 · XGBoost 0.894 · LightGBM 0.892 · GB 0.891; desvíos ≈ 0.04
- Ames, RMSE de test: XGBoost 24.4k · Gradient Boosting 26.8k · LightGBM 27.0k · Random Forest 27.5k · AdaBoost 36.2k
- Cuanto más cerca de 0 el RMSE, mejor; 25k significa errar ≈ 25 mil dólares por casa

Cuidado
Una sola partición no declara un ganador: en Heart las diferencias entre modelos caben dentro del desvío. Para una conclusión más sólida: validación cruzada anidada o varias semillas.

Chuleta, hiperparámetros y checklist

Lo que vas a ajustar en cualquier modelo de boosting

Función	Parámetros	Efecto si lo subes	Valor de partida
Capacidad	n_estimators (sklearn, XGBoost, LightGBM) · iterations (CatBoost)	más etapas: menos sesgo, más costo y más riesgo de sobreajuste	100–1.000 con early stopping
Paso	learning_rate (η)	más rápido, pero más ruidoso y menos estable	0.03–0.1
Árbol	max_depth · num_leaves (LightGBM)	más complejidad e interacciones	profundidad 2–6; hojas 15–31
Regularización	min_child_weight · min_data_in_leaf (min_child_samples) · reg_lambda (L2)	hojas más robustas y árboles más simples	L2 = 1 (el valor típico)
Aleatoriedad	subsample · colsample_bytree	valores < 1: más diversidad y menos sobreajuste	0.7–1.0
Parada	early_stopping_rounds · n_iter_no_change	más paciencia, más rondas	20–100

Función y parámetros: diapositiva 15. Los valores de partida son orientativos (ver notebook); el profesor aconseja entender cada regularización antes de meterla a un buscador automático.

Modelo	Sello distintivo	Categóricas	Velocidad	Riesgo principal
AdaBoost	Pondera casos difíciles	Codificar	Media	Ruido y outliers
Gradient Boosting	Gradiente de una pérdida	Codificar	Media-baja	Ajuste sensible
XGBoost	Regularización y sistema escalable	Depende de la implementación	Alta	Complejidad del tuning
LightGBM	Histogramas y leaf-wise	Nativas	Muy alta	Sobreajuste en datos pequeños
CatBoost	Pensado para categóricas (solo mencionado en clase)	Nativas, es su fuerte	—	—

Checklist para entrenar un modelo de boosting

	Paso	Cómo sé que lo hice bien
☐	Parte train/test (estratificado si es clasificación)	en regresión no se estratifica
☐	Preprocesa dentro del Pipeline	imputación y one-hot se aprenden solo con el train
☐	Empieza con una línea base	AdaBoost o un bosque, para saber qué hay que superar
☐	Fija η bajo y deja que early stopping elija n	con validación aparte del test
☐	Limita la complejidad del árbol	profundidad, hojas mínimas; cuidado con LightGBM en datos chicos
☐	Compara train vs test	una brecha grande es sobreajuste
☐	Incluye tu configuración base en la búsqueda	si no, la búsqueda puede elegir algo peor
☐	Reporta media ± desvío y compara bajo la misma partición	una sola partición no basta

Alertas del material, repaso y criterio de dominio

Diferencias entre materiales, preguntas clave y cuándo pasar a la siguiente clase

Alertas del material

Diapositivas y clase (verificados)

- **«AdaBoost = additive boosting»** (clase): el nombre viene de *Adaptive Boosting* (Freund y Schapire, 1997).
- **Diapositiva 16:** el título dice «cinco modelos», pero la tabla trae cuatro (falta CatBoost, que sí aparece en la diapositiva 18). La numeración impresa salta del 13 al 15: el archivo tiene 19 láminas y falta la «14/20».
- **«Gradient Boosting se detiene con tolerancia ≈ 0.001»** (clase): la teoría de optimización lo dice, pero `GradientBoosting*` de sklearn entrena exactamente n_estimators árboles salvo que actives `n_iter_no_change` (su tol por defecto es 1e-4).
- **«Boosting trabaja sobre y − F»**: eso es el gradiente negativo solo con pérdida cuadrática. Y «menor η, mejor» solo si aumentas los árboles.

Notebook del profesor (Ejemplo Boosting.ipynb)

- `LGBM…(subsample=0.8)` sin `subsample_freq` ignora el submuestreo: en el notebook guiado se comprueba. XGBoost sí lo aplica.
- El GB de Ames usa `loss='huber'`, pero se evalúa y optimiza con RMSE (pérdida cuadrática): objetivo y métrica no coinciden.
- La «importancia» compara cosas distintas: en XGBoost es por **ganancia**; en LightGBM, por **número de cortes** (*split*).
- En clase se dijo que se eliminaban Latitude y Longitude de Ames; el notebook entregado no lo hace.
- Las dos comparaciones finales usan una sola partición (76 pacientes de test): que AdaBoost «gane» por un paciente no es evidencia.

Transcripción con ruido

«bugging» (bagging), «Roundoff Forest» (Random Forest), «Caldus» (CatBoost), «XzBus/Xtbus» (XGBoost), «Light DBM» (LightGBM), «KDNN» (KNN), «Cycle Learn» (scikit-learn), «Nameless Housing» (Ames Housing), «KERX» (Heart). Bucles de Whisper en las líneas 216–244 y 1337–1341; comentario ajeno a la clase en las líneas 508–512.

Preguntas de repaso

#	Pregunta	Respuesta corta
1	¿Qué cambia entre AdaBoost y Gradient Boosting?	AdaBoost pasa el error como pesos de los casos; Gradient Boosting como el gradiente de la pérdida (residuos si es cuadrática).
2	¿Por qué η y n_estimators interactúan?	Menor η da pasos chicos, así que hacen falta más árboles para llegar al mismo punto.
3	¿Cuándo priorizarías LightGBM?	Muchos datos y variables, o presupuesto de cómputo ajustado; con datos pequeños, con cuidado.
4	ε = 0.5 en una ronda de AdaBoost, ¿qué α recibe?	$\alpha = \frac{1}{2}\ln(1) = 0$: el árbol no aporta al voto.

Criterio de dominio para pasar a la siguiente clase

- ☐ Hago a mano una ronda de AdaBoost: ε, α y nuevos pesos
- ☐ Hago a mano una etapa de Gradient Boosting con error cuadrático
- ☐ Explico por qué η y el número de árboles se ajustan juntos
- ☐ Uso early stopping con un conjunto de validación propio
- ☐ Distingo XGBoost (por nivel, regularizado) de LightGBM (por hoja, histogramas)
- ☐ Resuelvo los bloques 1 a 9 del cuaderno con al menos 80% de aciertos